

Lies Ben
Mayers Christophe

Thrust Vector Control System and Flight Stabilisation for Model Rockets

Abstract

Rocketry is an engineering discipline concerned with the design, construction, and launch of rockets at both professional and amateur levels [1]. Due to its modular nature, it provides an effective framework for introducing engineering concepts at an early stage, allowing increasing levels of complexity to be incorporated across successive projects.

A first model rocket was launched in 2023 [2] and was equipped with an Arduino-based flight computer for data acquisition. Post-flight analysis revealed significant issues, including an unstable, highly elliptical flight path and limited usefulness of the collected data. These shortcomings motivated the development of the present project.

Building upon the earlier design, a new model rocket was designed, constructed, and simulated. Improvements included a smaller body tube, a higher-performance flight computer, and the integration of a flight guidance and stabilisation system based on thrust vector control. The primary objectives were achieved; however, the results also revealed new limitations, providing clear direction for future design iterations.

Index terms- Rocketry, guided flight, thrust vector control, flight stabilisation

Contents

1	Introduction	6
1.1	Domain	6
1.2	Scientific deliveries	6
1.3	Technical deliveries	6
2	Methodology	7
2.1	Tools and prerequisites	7
2.2	Printed Circuit Board (PCB)	8
2.2.1	Design	8
2.2.2	Production	9
2.3	Thrust-Vector-Control System (TVC)	9
2.3.1	Design	9
2.3.2	Production	9
2.4	Reaction wheel	11
2.4.1	Design	11
2.4.2	Production	13
2.5	Rocket body and nosecone	15
2.5.1	Design	15
2.5.2	Production	16
2.6	Software	16
2.6.1	Madgwick Filter [14]	16
2.6.2	State Machine	17
2.6.3	LQR Controller	18
2.7	Simulations	19
3	Results	21
3.1	Printed Circuit Board (PCB)	21
3.2	Thrust-Vector-Control System (TVC)	21
3.3	Reaction Wheel	22
3.4	Body and nosecone	23
3.5	Simulations	23
3.6	Software	25
4	Conclusion	25

List of Figures

1	Final board designed in EAGLE	8
2	Final assembled PCB	9
3	First TVC Mount prototype	10

4	Curve Fitting of the Y TVC Servo	10
5	Reaction wheel	12
6	Casing for the reaction wheel and PCB	12
7	Test setup	13
8	Early rocket body design	15
9	Nosecone	15
10	Overview of the state machine	17
11	Overview of the whole SIMULINK simulation	19
12	Simulation of the rocket dynamics	19
13	Simulation of the LQR Controller	20
14	Converting torque to TVC angles	20
15	Final TVC	21
16	Finished reaction wheel	22
17	Final assembled rocket	23
18	Torque applied to the rocket	24
19	Simulation results	24
20	Software results	25

1 Introduction

Aerospace engineering has seen drastic advancements and an increasing public interest in Luxembourg. With the Luxembourg Space Agency (LSA) pushing the frontiers of international politics into an extra-planetary dimension in pursuit of a “commercial space hub”[3], the question of a sustainable rocket launching infrastructure rises naturally. The core problem of a space economy and the ability to utilise the associated vast resources is a logistical one. Transportation costs are currently not sustainable[4]. Reducing overall cost per launch is hence a necessary step for future space travel that can be achieved by optimizing the most cost inducing sections, like booster stages. Boosters are rocket engines that help with the initial thrust of a rocket to escape Earth’s gravity until reaching low orbit [5]. After expending all available fuel, they are cut off [6] and fall back to earth. The possibility of reusing these booster stages for future missions would substantially reduce operational costs [7] and could be achieved by making them land themselves, reducing potential damages. This work focuses on replicating a mandatory capability towards this novelty, namely controlled flight, and documenting it.

1.1 Domain

This project can be placed into the engineering domain. Main problems will revolve around correct algorithm tuning, managing components and the operating speed of the flight computer. Connecting so many electrical components to a development platform usually results in unpredictable behavior and requires a lot of careful preparation. This will be solved using a printed circuit board. The chosen algorithm must be adapted to the rocket, simulated and tuned which is closer to an art form than a science. However, the most important aspect is the operating speed of the development board which must keep up with mid-flight-calculation and communication. These parameters are most decisive for the success of this project.

1.2 Scientific deliveries

The scientific deliveries for this project are a deepened understanding of rocketry and preparing for more complex future projects. Rocket science is renowned to demand precision and a lot of knowledge. To attain a professional, or even an amateur level, can take years and depend on a lot of dedication, like any other skill. With the finalisation of this project, perseverance is proven and a contribution to public awareness is made. Potentially inspiring others to become scientists themselves would be one of the most preferable deliveries: “Not everyone can become a great [scientist], but a great [scientist] can come from anywhere”. (Ratatouille, 2007)

1.3 Technical deliveries

The technical deliveries are more dispersed and build up to a fully functioning model rocket with a directional guidance system. The hardware consisting of the printed circuit board (PCB),

the thrust-vector-control (TVC) system, the reaction wheel, the nose cone with its parachute release mechanism and the rocket body. The software will be a directory of files responsible for the communication between components, collecting information, controlling servo and brushless motors, and the flight algorithm.

2 Methodology

2.1 Tools and prerequisites

Since this project encompasses the entire production process of a rocket, a wide range of technical skills and tools is required:

- **Simulink** is the software used to simulate the flight algorithms performance and later tune the necessary parameters in the program on the development board. A rocket computer alone is unaware that it is a flying object and only reacts within the rules of its programming. If one were to use the exact same algorithm for a 2m and a 20 cm long rocket, one would most likely have drastically different results. Flight algorithms are not “add-on solutions” and must be tested and adjusted. This fine tuning allows to adapt the algorithm to the rocket’s specific requirements. Understanding how Simulink works is hence crucial for this project.
- **Autodesk Fusion360** is the computer-Aided-Design (CAD) platform used for 3d modelling. 3D modelling is necessary to visualise and later easily, qualitatively and efficiently try out hardware components for the rocket. This skill set is hence not only a must for this project, but for any future engineer.
- **GitHub** is the chosen developer platform that helps to organise, manage and store our files for this project. It is beneficial, but not mandatory since it mostly just improves team related workflow.
- **Arduino/C++/Matplotlib/Jupyter Notebook** are the programming languages used for this project because of all the algorithms and scripts for the electronic components. Matplotlib and jupyter notebook were mostly used to do some quick calculations and TVC gimbal curve fitting. The entire software relies on understanding these programming languages.
- **Autodesk Eagle** is an electronic design automation software (EDA) that helps to design PCBs. As this project revolves around a custom printed circuit board, knowing the physical behaviour of the components and being familiar with this tool is recommended.
- **Prusa/Cura Slicer and Bambulab PS1 printer** are the next natural step after the 3D modelling program. The slicer is used to prepare the 3D model for printing. Most parts of this project require high precision and durability. Access to good printers and a general comprehension of this process is therefore compulsory.

- **Hand tools and painting equipment** are also required. All secondary tasks, like the assembly of the PCB or painting the rocket body need tools, like a soldering iron or spray paint.

2.2 Printed Circuit Board (PCB)

The PCB is the heart of the entire rocket. It is connected to an 11.1V battery and controls all the actuators. Besides that, it gathers important sensor data, logs it onto an SD Card and has sensor fusion to predict and correct the rocket's position.

2.2.1 Design

The design process mostly began with the question of which information would be needed to keep a rocket upright. We settled on a BNO055 inertial measurement unit (IMU) sensor to obtain the orientation and a BMP390 barometer to determine the altitude. Then, only the actuator side needed to be covered. To accomplish this, three terminal blocks were used to connect the X and Y servos of the TVC and the servo controlling the parachute ejection. For visual and audible feedback, a bright RGB LED and a buzzer was chosen. As most of the components required 3.3V or 5V, a voltage converter from 11.1V to 5V was implemented. The most important component of the PCB was a Teensy 4.1 acting as main MCU. It is known for its incredible speed and even having an SD Card reader/writer already onboard, outperforming the more commonly used Arduino Uno by far.

As the last step we used EAGLE to draw a simple schematic and create a board connecting all previously chosen components. Additionally, some headers for debugging and a reaction wheel was added.

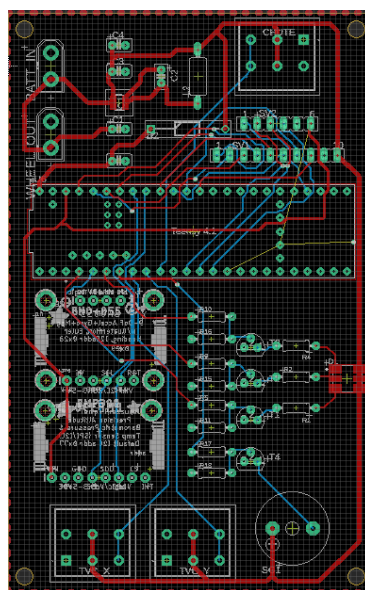


Figure 1: Final board designed in EAGLE

2.2.2 Production

The files of the PCB were sent to Eurocircuits [8] from which the later on finished boards were received. All components were then soldered onto it and tested with a voltmeter to prevent unexpected future errors. To exemplify, it was through this method immediately noticeable that an RGB LED was soldered the wrong way onto the PCB or the voltage converter design flawed. The voltage converter was then later replaced by the brushless motor's electronic speed controller which luckily also had the necessary 5V power distribution.

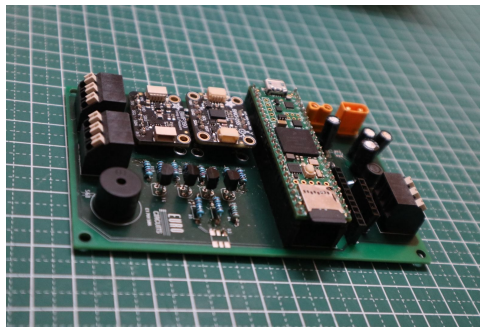


Figure 2: Final assembled PCB

2.3 Thrust-Vector-Control System (TVC)

The TVC can be simply described as a two axis gimbal which holds a solid rocket motor in place and allows to precisely control the motors orientation through two servos. Without it, stabilizing a rocket would be impossible.

2.3.1 Design

For the design we looked for online inspiration and quickly found some already existing solutions [9] which seem to work quite well. After adapting the design to the required parameters of the chosen rocket body in Fusion360, the first prototype was made.

2.3.2 Production

The TVC mount was fully 3D printed in PLA and then assembled with screws and servo linkages until we end up with the following:



Figure 3: First TVC Mount prototype

Having the physical model ready, it was then essential to determine a function that converts servo angle to gimbal angle for both axis. Two approaches were trialed to achieve this:

Engineering approach: For this approach we strapped an inertial measurement unit (IMU) to the gimbal, sent known angles to the servos and measured the resulting gimbal angle with the IMU. This process was repeated several times for redundancy. Afterwards, one would only need to do some curve fitting with Matplotlib/Jupyter Notebook and get an appropriate function.

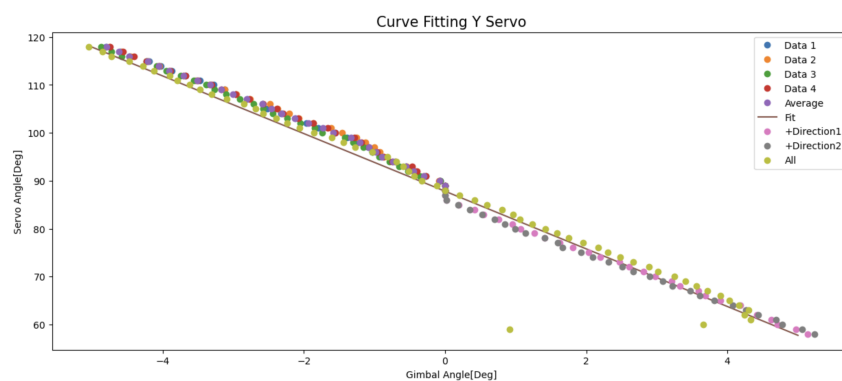


Figure 4: Curve Fitting of the Y TVC Servo

Mathematical approach: For this approach we used a mathematical model developed by the YouTuber Nicola Gaiani [10]. One would need to enter the specific parameters of the TVC mount and then get for each servo angle input an appropriate gimbal angle output. This process required curve fitting again.

Source code 1: Mathematical model of the TVC mount

```

1 import math
2
3 # Constants (in mm)
4 l1_s1 = 10.5      # length of servo attach
5 l2_s1 = 32.5      # bar length
6 l3_s1 = 42.5      # distance from pin to bar on the 3D printed part
7 Cx_s1 = 30.0      # x distance between servo wheel and mount pin
8 Cy_s1 = -29.0     # y distance
9
10 import numpy as np
11
12 def m_to_s1(a):
13     a = np.radians(90-a)
14
15     px = Cx_s1 + l3_s1 * np.cos(a)
16     py = Cy_s1 + l3_s1 * np.sin(a)
17     d = np.sqrt(px**2 + py**2)
18
19     x1 = (l1_s1**2 + d**2 - l2_s1**2) / (2 * d)
20     y1 = np.sqrt(l1_s1**2 - x1**2)
21
22     ang = np.arctan2(py, px)
23
24     x2 = x1 * np.cos(ang) - y1 * np.sin(ang)
25     y2 = x1 * np.sin(ang) + y1 * np.cos(ang)
26
27     return np.degrees(np.arctan2(y2, x2))
28

```

2.4 Reaction wheel

The reaction wheel tries to conserve the angular momentum of the rocket while not using any fuel. It tries to counter rotate any undesired rotational spin, caused by air drag for example, acting as a small support system for the thrust vector control system. By keeping the rotational speed low, or null at best, it could prevent a potential coning motion, usually referred to as precession. In this project, it consists of an electronic speed controller (ESC), a brushless motor and the reaction wheel.

2.4.1 Design

The design phase mostly revolved around finding an acceptable hardware solution for a reaction wheel, finding a good production method and deciding on its placement in the rocket.

Originally, the reaction wheel was supposed to be out of lightweight aluminum to maximize the power limits of the brushless motor, but the addressed milling company made us reconsider. Consequently, it was 3D printed. To achieve the highest moment of inertia the reaction wheel got the form of a ring with radial support to a central hub that connects directly to the brushless motor [11]. To compensate for the missing weight, screw holes were added for later weight addition.

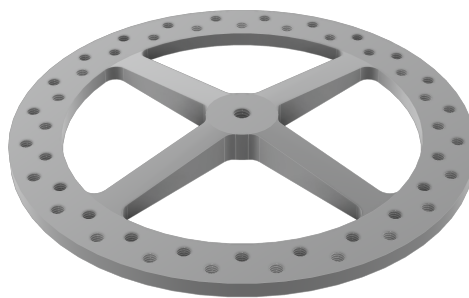


Figure 5: Reaction wheel

To guarantee the safety of the internal rocket body and simultaneously give the brushless motor a fixed place, a casing was designed for it. However, the casing sealed the motor from the PCB and entailed the electronics to run between the inner rocket body wall and the casing.

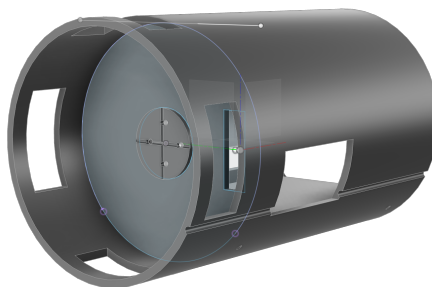


Figure 6: Casing for the reaction wheel and PCB

2.4.2 Production

The production process describes the practical realization of the reaction wheel and its addition to the PCB. It began with 3D printing the reaction wheel and adding screws onto it. Afterwards, the brushless motor was tested while attached to the PCB.

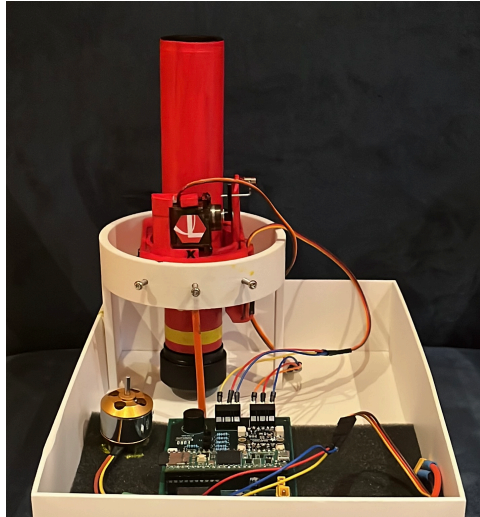


Figure 7: Test setup

In these coding experiments, the electronic speed controller didn't support bi-directional spin. After projecting the problem into the domain of model cars, it could have been solved with an H-bridge, allowing switching the polarity of the brushless motor with relays. This task was not foreseen, implied a redesign of the power supply and remodeling of all involved components. The easiest solution was however to replace it with a new bidirectionally supported ESC, even though it was monetarily more expensive. The latter solution was implemented.

For improved movement, we added a PID controller to the reaction wheel [12]. A PID controller is a control algorithm that works with a continuous feedback loop for a correct adjustment around a fixed setpoint. In this project, this fix set point can be interpreted as a state, namely the rocket not rotating around its own axis. This correction revolves around magnitude of the error between the setpoint and the actual value. This controller projects this error into the near future and remembers how it was in the past. Based on this information it tries to decide to adjust the motor speed accordingly. This lowers the risk of overshooting and consequently making too harsh movements in this time sensitive operation.

Source code 2: PID controller implementation for reaction wheel

```

1 void MotorController::update() {
2     // 1. Read gyro (Y-axis rate)
3     imu::Vector<3> gyro = bno.getVector(Adafruit_BNO055::↔
        VECTOR_GYROSCOPE);
4     measuredRate = gyro.y();
5 }

```



```
6 // 2. Time step
7 unsigned long now = millis();
8 float dt = (now - prevTime) / 1000.0;
9 prevTime = now;
10
11 // 3. PID
12 float error = setpoint - measuredRate;
13 integral += error * dt;
14 float derivative = (error - prevError) / dt;
15 prevError = error;
16
17 float pidOutput = Kp * error + Ki * integral + Kd * derivative↵
18 ;
19
20 // 4. Compute throttle
21 int throttle = computeThrottle(error, pidOutput);
22 esc.writeMicroseconds(throttle);
23
24 // 5. Debug
25 Serial.print("Y rate: "); Serial.print(measuredRate);
26 Serial.print(" | Error: "); Serial.print(error);
27 Serial.print(" | PID: "); Serial.print(pidOutput);
28 Serial.print(" | Throttle: "); Serial.println(throttle);
29 }
```

After testing completed successfully, no further problems arose. The reaction wheel was added to the casing and the ESP finally soldered to the PCB. We then used 2 component glue to fix the casing into the rocket body.

2.5 Rocket body and nosecone

The rocket body contains all the electronics. The nosecone is responsible for the parachute release.

2.5.1 Design

The design phase of the rocket body and nosecone covers the decision for the size, appearance and the parachute mechanism.



Figure 8: Early rocket body design

The most influential factor for the decision of the rocket body was convenience. It should be large enough to justify a TVC system, but not too large to unnecessarily waste space. A tube with a diameter of 10 cm and length of 100 cm was chosen. The nose cone must have a reliable parachute release mechanism and fit our personalised rocket body. Taking inspiration from the Youtube Channel Air Command Rockets [13] it was hence opted to 3D model it and add an empty chamber that opened with a servo motor. This servo motor would hold a plate that could fall off and open the chamber, allowing the parachute to simply fall out.



Figure 9: Nosecone

For the paint job, it was decided to go for a bright red with yellow and dark elements imitating a famous Formula 1 car design.

2.5.2 Production

The production process included the rocket body and nosecone modifications. On arrival, the rocket body was painted with spray cans and then cut in half. Firstly, it received a black coating and then red stripes. At the same time, we painted the nosecone completely red. The cord was attached to the parachute, folded, put into the nosecone chamber and then sealed with the door. After putting in all the electronics, the lower half contained the TVC system and PCB. The upper part consisted out of the reaction wheel and the nosecone. At last, we reattached the 2 halves.

2.6 Software

The onboard flight software is responsible for real-time state estimation, flight-phase management, and thrust vector control during powered ascent.

The software is structured into three main layers:

1. Sensor acquisition and attitude estimation,
2. State machine governing flight phases, and
3. a control layer computing thrust vector commands based on the estimated state

2.6.1 Madgwick Filter [14]

Accurate attitude estimation is essential for thrust vector control, as control commands depend directly on the rocket's orientation. Gyroscopes provide smooth short-term orientation but suffer from drift, while accelerometers provide an absolute reference to the gravity direction but are sensitive to dynamic accelerations.

The Madgwick filter fuses both sensor types in a computationally efficient manner, making it well suited for real-time embedded applications. Orientation is represented using quaternions [15], avoiding gimbal locks. Gyroscope measurements are used to propagate orientation through quaternion integration, while accelerometers are used to correct long-term drift.

The correction is obtained by comparing the measured gravity direction with the direction predicted by the current orientation estimate. The resulting misalignment is expressed as an error function in quaternion space and its gradient provides a correction direction. The estimated quaternion derivative is given by:

$$\dot{\mathbf{q}}_{\text{est}} = \dot{\mathbf{q}}_{\omega} - \beta \frac{\nabla f}{\|\nabla f\|} \quad (1)$$

Where β controls the influence of the accelerometer-based correction. In our case this value is set very low as the rocket represents a highly dynamic environment.

2.6.2 State Machine

The flight software is organized as a finite state machine to ensure deterministic and robust behavior throughout the flight. Each state encapsulates a distinct flight phase with well-defined actions and transition conditions. Separating the flight into phases also improves safety, since time-critical control tasks are only enabled when required.

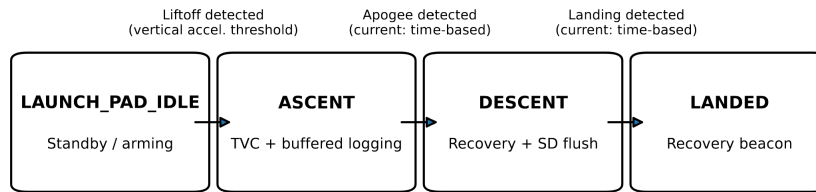


Figure 10: Overview of the state machine

LAUNCH_PAD_IDLE (pre-launch) In the `LAUNCH_PAD_IDLE` state, the flight computer remains in a safe standby mode on the launch pad. Sensors are active and the system continuously evaluates a liftoff condition using the measured vertical acceleration. Once liftoff is detected, the system timestamps the start of flight and records a corresponding entry in the event log.

Transition to ASCENT: Liftoff detected (vertical acceleration threshold exceeded).

ASCENT (powered flight / TVC active) During `ASCENT`, thrust vector control is enabled and the software runs two periodic tasks with different update rates. The control loop executes at 100 Hz and performs attitude estimation, control law evaluation, and servo command output. In parallel, a health monitoring and logging task runs at 25 Hz. To avoid non-deterministic latency introduced by SD card writes, high-rate flight data is buffered in the Teensy flash memory during ascent. Discrete events (e.g., liftoff, apogee) are stored in a separate event file together with timestamps.

Transition to DESCENT: Apogee detected (current implementation: time-based criterion).

DESCENT (recovery phase) After apogee, the system transitions to `DESCENT` and the parachute is ejected. Since strict real-time control is no longer required, buffered flight data stored in flash is flushed to the SD card during descent. Logging is finalized and closed as well as flight control actions are disabled.

Transition to LANDED: Landing detected (current implementation: time-based criterion).

LANDED (post-flight) In the terminal LANDED state the system enters a safe post-flight mode. A recovery beacon (buzzer and LED) is activated to facilitate locating the rocket.

2.6.3 LQR Controller

During powered ascent, the rocket is stabilized using a Linear Quadratic Regulator (LQR) acting on the rotational dynamics of the vehicle. The system is linearized about the nominal upright flight condition, and the state vector is defined as

$$\mathbf{x} = [\boldsymbol{\theta} \quad \boldsymbol{\omega}]^T, \quad (2)$$

where $\boldsymbol{\theta}$ denotes small attitude deviations from the vertical reference and $\boldsymbol{\omega}$ the corresponding angular rates.

The LQR computes a state-feedback control law of the form

$$\boldsymbol{\tau}_{\text{cmd}} = -\mathbf{K}\mathbf{x}, \quad (3)$$

where $\boldsymbol{\tau}_{\text{cmd}}$ is the demanded control torque about the rocket's center of mass. The gain matrix $\mathbf{K}$ is obtained by minimizing the quadratic cost function

$$J = \int_0^{\infty} (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \boldsymbol{\tau}_{\text{cmd}}^T \mathbf{R} \boldsymbol{\tau}_{\text{cmd}}) dt. \quad (4)$$

The weighting matrices $\mathbf{Q}$ and $\mathbf{R}$ penalize attitude deviations and excessive control torque, respectively, allowing a systematic trade-off between stability and control effort.

The demanded torque is subsequently converted into thrust vector deflection commands using the known thrust magnitude and lever arm of the thrust vector control mechanism. The LQR controller is active exclusively during the ascent phase and relies on the attitude estimate provided by the Madgwick filter.

2.7 Simulations

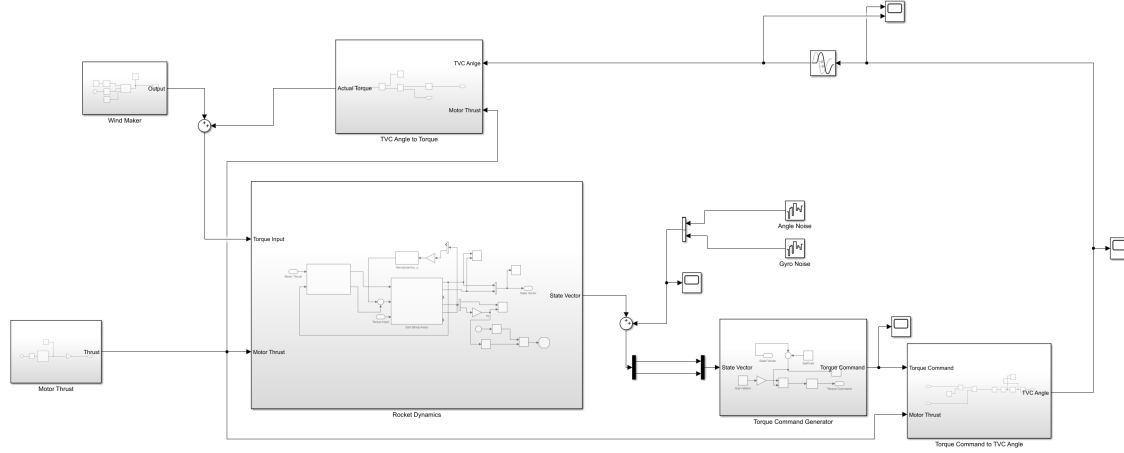


Figure 11: Overview of the whole SIMULINK simulation

A nonlinear simulation environment was developed in SIMULINK to validate the proposed state estimation and control architecture prior to flight testing. The simulation captures the rotational dynamics of the rocket, thrust vector control actuation, sensor noise, and external disturbances, while maintaining a structure that closely resembles the onboard flight software. This allows controller parameters and state machine logic to be evaluated under realistic conditions before deployment.

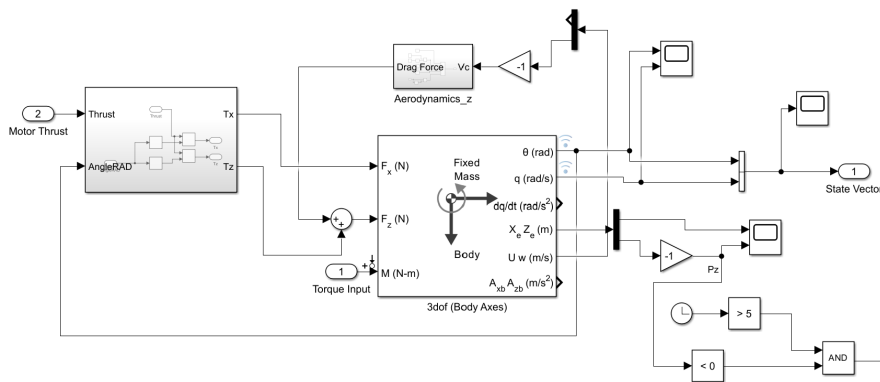


Figure 12: Simulation of the rocket dynamics

Rocket Dynamics Model The rocket is modeled as a rigid body rotating about its center of mass. The rotational dynamics are governed by Euler's equation:

$$I\dot{\omega} = \tau_{TVC} + \tau_{dist},$$

where I denotes the moment of inertia, τ_{TVC} the control torque generated by thrust vectoring, and τ_{dist} external disturbance torques.

The angular rate is integrated to obtain the attitude state, which forms the basis for feedback control. Motor thrust is provided as an external input to the dynamics block, allowing the simulation to capture thrust variations over time.

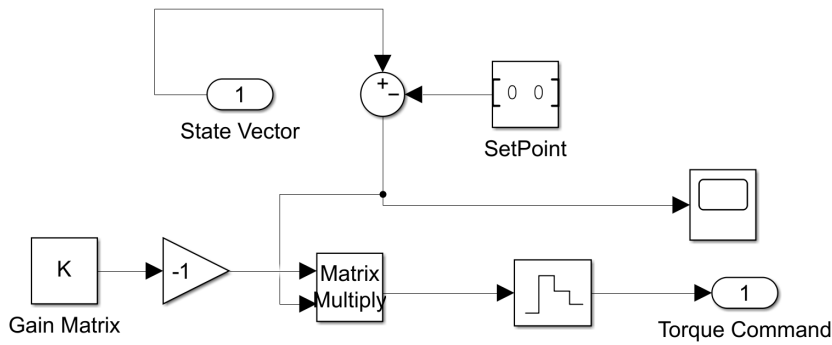


Figure 13: Simulation of the LQR Controller

Thrust Vector Control and Actuator Mapping The LQR controller outputs a demanded control torque about the rocket body axes. This torque is converted into a thrust vector deflection angle using the known motor thrust magnitude and moment arm of the TVC mechanism. The resulting TVC angle represents the actuator command applied to the motor mount.

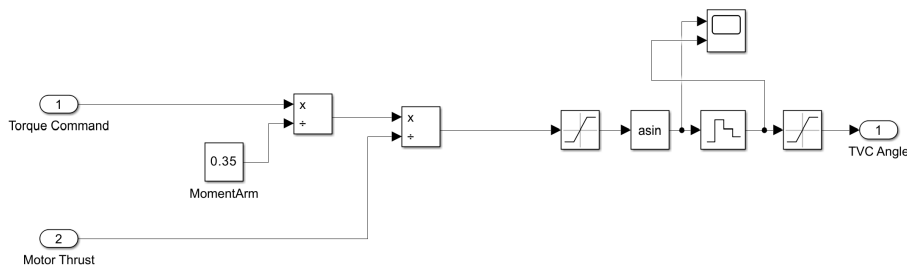


Figure 14: Converting torque to TVC angles

Within the simulation, this mapping takes into account actuator limits (max deflection angle, actuator delay, etc) as well as flight computer limits (control loop runs at 100Hz)

Sensor Noise and Disturbances To evaluate robustness, sensor noise and external disturbances are explicitly included in the simulation. Gaussian noise is added to both the measured attitude and angular rate signals to emulate inertial sensor imperfections.

Additionally, external disturbance torques are injected into the system to represent aerodynamic effects such as wind gusts. These disturbances act directly on the rotational dynamics and allow the controller's disturbance rejection capability to be assessed.

3 Results

In the following subsections, each component is analysed and discussed. The discussions will revolve around expected behaviour, the accuracy of the simulation and suggested improvements for the next project. *Disclaimer: As there were two failed launches our results only rely on ground tests.*

3.1 Printed Circuit Board (PCB)

The PCB works perfectly as expected besides the voltage converter being replaced by the motor driver. Structurally one could improve the terminal blocks because cables tend to get stuck in them and are therefore hard to remove. In addition one could have made appropriate outputs for the reaction wheel and reduce its size as it is pretty chunky[16].

3.2 Thrust-Vector-Control System (TVC)

The TVC fulfills its purpose but there are a lot of things to improve. First of all, it was printed in PLA which is structurally weak and sensitive to high temperature. Better would have been materials like ABS or nylon. Secondly, the motor plug at the top was printed with 30% infill and immediately ripped open when firing the rocket motor and therefore the reason for our first failed launch. Better would have been a 100% infill. Finally, the servos are fine when choosing a rocket motor with a max thrust of around 60N but above that we would have to look for stronger servos.



Figure 15: Final TVC

3.3 Reaction Wheel

The reaction wheel works as intended and can serve as a small support system. It is capable of counter acting the rotational movement the rocket could experience, lowering the risk of coning motions.

In contrast to the TVC system however, the range of referenced capabilities can be questioned because of a lack of calculations and simulations. Most parameters, like weight, motor strength and the necessity of a PID controller are estimated and not part of the simulations. It is hence a possible sources of error and can, in the worst case, cause the opposite of the desired effect.

Even if the reaction wheel has been successfully made, it will not be used in the launch since it could taint the following evaluation of the simulations too much. It will hence unfortunately only be additional weight. The reason for it not being implemented in the simulations is the difference in the time of completion of the components. The TVC System was the first realised part of the rocket and took months for proper simulation and testing. The idea and decision to add a reaction wheel came much later. It serves now as a prototype and can be considered in the next project from the very beginning.



Figure 16: Finished reaction wheel

3.4 Body and nosecone

As there have not been any strong modifications to the rocket body, there are not many results in that regard. The desired visual appeal has been achieved and the nosecone works reliably. Besides the parachute opening midair, there are no concerns.



Figure 17: Final assembled rocket

3.5 Simulations

The rotational dynamics used for LQR design are described by the linear state-space model

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\tau_{\text{cmd}}, \quad (5)$$

with the state vector

$$\mathbf{x} = \begin{bmatrix} \theta \\ \omega \end{bmatrix}.$$

The system matrices are given by

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ \frac{1}{I} \end{bmatrix} = \begin{bmatrix} 0 \\ 11.6686 \end{bmatrix}, \quad (6)$$

where the moment of inertia is $I = 0.0857 \text{ kg m}^2$.

The LQR weighting matrices are chosen as

$$\mathbf{Q} = \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{R} = 2.5. \quad (7)$$

Solving the algebraic Riccati equation yields the optimal state feedback gain

$$\mathbf{K} = [2.0000 \quad 0.8619], \quad (8)$$

resulting in the control law

$$\tau_{\text{cmd}} = -\mathbf{K}\mathbf{x}. \quad (9)$$

By applying random torque to the rocket over a period of time (simulating wind gusts or other disturbances) like shown in the following graph:

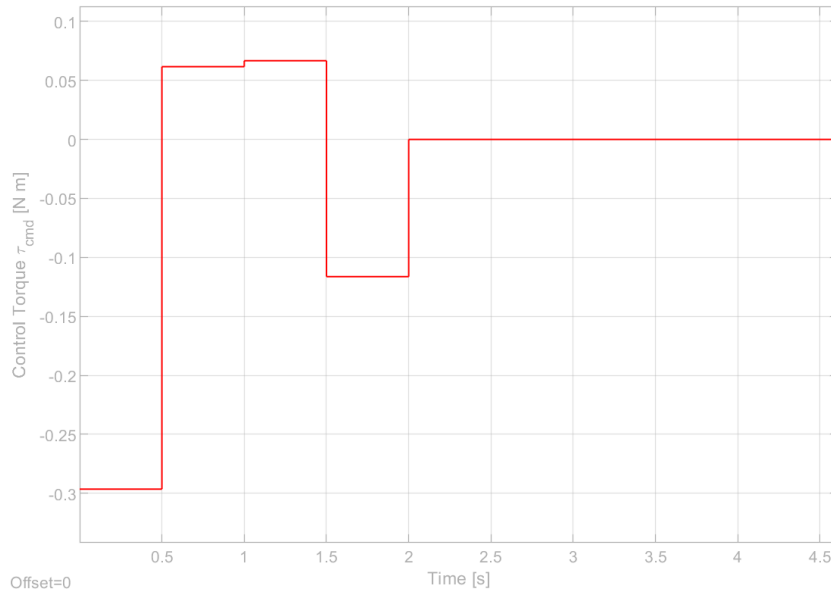


Figure 18: Torque applied to the rocket

We come to the following results:

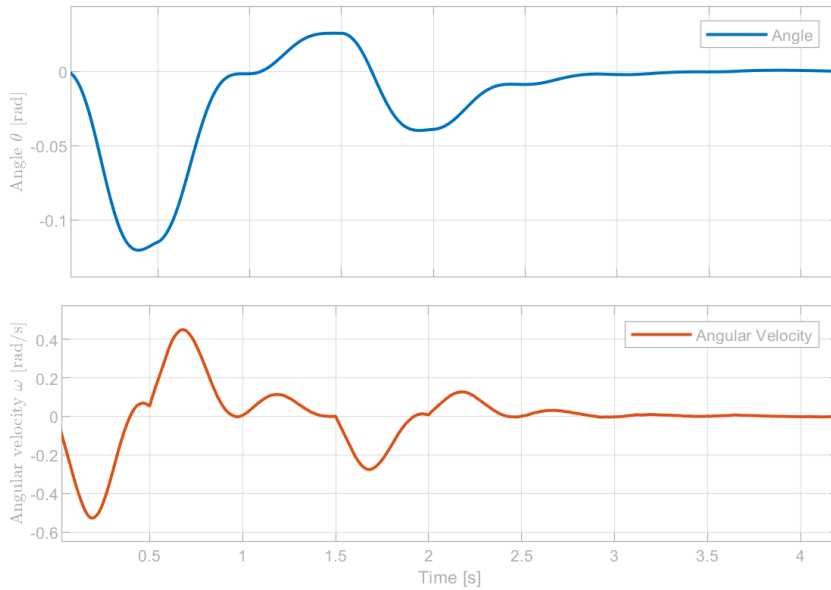


Figure 19: Simulation results

Figure 19 shows the resulting closed-loop attitude and angular rate response. Despite the applied disturbance torques, the controller maintains stability and rapidly drives both the attitude deviation and angular velocity back toward zero. The response exhibits well-damped behavior with no sustained oscillations, indicating effective disturbance rejection.

The peak attitude deviation remains limited, and angular rates stay within acceptable bounds for the thrust vector control system. This demonstrates that the selected LQR gains provide a

favorable balance between disturbance rejection performance and control effort, validating the controller design.

3.6 Software

By feeding our flight software with sensor data from the simulation we could check if the whole system acts as expected. Placing an IMU on the top of the TVC mount we get the following results:

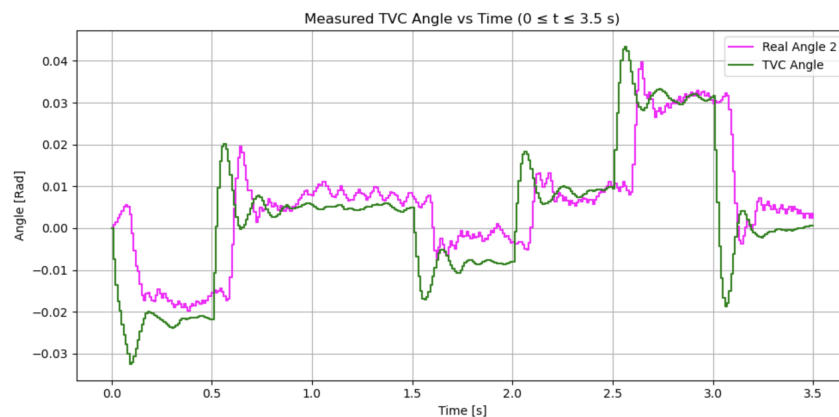


Figure 20: Software results

Figure 20 compares the simulated TVC angle (in green) with the measured TVC angle (in magenta) obtained during experimental testing. The measured response closely follows the simulated trajectory over the entire time interval, indicating that our software works perfectly.

A consistent time offset between the commanded and measured TVC angle is visible and corresponds to the servo actuation delay of approximately 0.09 s , which is already explicitly modeled in the simulation.

Remaining discrepancies between the simulated and measured TVC angles are attributed primarily to the measurement setup. As stated the TVC angle is inferred from an inertial measurement unit mounted on top of the TVC mechanism, which introduces additional dynamics and measurement noise.

4 Conclusion

To conclude, this project was a total success. In 6 months, the entire process of designing, building and simulating a model rocket was achieved. It was a great opportunity to improve necessary skills for future projects, like 3D modelling, printing, painting, soldering, simulating and, most importantly, teamwork. A deeper understanding of rocketry has also been accomplished by discovering the Madgwick filter allowing for accurate sensor fusion, the importance

of simulations for component expectations and the gained knowledge on solving the problem for controlled flight. The project hence resulted in a fully functioning, painted and simulated rocket.

When comparing the previously made model rocket in 2023, major differences can be noted. Firstly, the PCB was a huge improvement in regards to cable management and testability. It saved a lot of time and, of course, was visually more appealing. Secondly, it should be impossible with the newly designed nosecone for the parachute to remain stuck in the rocket, like the last time. Lastly, the TVC system could solve the biggest problem yet encountered: the rocket drifting in the wrong direction. This was the reason why the last rocket recovery had become fairly time intensive. All these improvements make this rocket a milestone of a project. This journey is however far from over and every skill and all the experience will be carried into the next projects, just like before.

With all the calculations, simulations, expectations and waiting, there is only one thing left to do and we are sincerely looking forward to it. This project was presented to you by Ben and Christophe and we are ready for our third attempt.

References

- [1] <https://www.collinsdictionary.com/dictionary/english/rocketry> accessed on 12 January 2026.
- [2] <https://www.youtube.com/watch?v=nfHlhmoQnFs> accessed on 27 January 2026.
- [3] <https://spacewatch.global/2018/09/luxembourg-space-agency-open-for-business-as-new-space-hub/> Accessed on 28 January 2026.
- [4] <https://www.bbc.com/news/articles/cjewvpkw7weo>
- [5] <https://www.nasa.gov/wp-content/uploads/2012/07/rockets-educator-guide-20.pdf> Accessed on 28 January 2026.
- [6] [https://en.wikipedia.org/wiki/Booster_\(rocketry\)](https://en.wikipedia.org/wiki/Booster_(rocketry)) Accessed on 28 January 2026.
- [7] <https://www.bbc.com/news/science-environment-24331860> Accessed on 28 January 2026.
- [8] <https://www.eurocircuits.com/> Accessed on 2 February 2026.
- [9] <https://www.printables.com/model/9920-k-9-rocket-thrust-vector-control-gimbal-v8/related?lang=de> Accessed on 2 February 2026.

- [10] https://www.youtube.com/@nicola_gaiani Accessed on 2 February 2026.
- [11] https://link.springer.com/book/10.1007/978-3-662-59886-3?utm_source=chatgpt.com (S. 290). Accessed on 9 February 2026.
- [12] https://en.wikipedia.org/wiki/Proportional%E2%80%9393integral%E2%80%9393derivative_controller Accessed on 28 January 2026.
- [13] <https://www.youtube.com/watch?v=COkh5ykc3Y> Accessed on 9 February 2026.
- [14] <https://medium.com/@k66115704/imu-madgwick-filter-explanation-556fbe7f02e3> Accessed on 9 February 2026.
- [15] <https://lisylarus.github.io/blog/posts/introduction-to-quaternions.html> Accessed on 9 February 2026.
- [16] <https://www.youtube.com/watch?v=vx5vpG6jEXI> Accessed on 9 February 2026.
- [17] ChatGPT was used to check over the text and correct. In the abstract it was used to rephrase. It was also used to generate code.
- [18] <https://github.com/C442/Rocket> The Github repository designated to this project 

